



Universiteit
Leiden
The Netherlands

Web Security I

Lecture Handout

Eleftheria Makri

2025-2026

Web Security

Web applications are mostly written in memory safe languages and thus do not suffer from buffer overflow vulnerabilities. However, many underlying issues in web security are similar: untrusted, uncontrolled data can be treated by a misguided web app as code. Modern web, in addition to those old issues, has brought some new ones: we now have mobile code executed in the context of the browser. This increases the attack surface.

1. Web Client-Server Interaction

The Client-server model is a distributed application structure that partitions tasks or workloads between the providers of a **resource** or service, called **servers**, and service requesters called **clients**. In the client-server architecture, when the client computer sends a request for data to the server through the internet, the server accepts the requested process and delivers the data packets requested back to the client. Clients do not share any of their resources. Examples of the Client-Server Model are Email, World Wide Web, etc., wherein the **resources** could be webpages, or other content. Resources are identified via a URL (Universal Resource Locator). Resources can be static (html files) or dynamic (php files).

Client: When we say the word Client, it means to talk of a person or an organization using a particular service. Similarly in the digital world, a Client is a computer (Host) i.e., capable of receiving information or using a particular service from the service providers (Servers).

Servers: Similarly, when we talk about the word Servers, It means a person or medium that serves something. Similarly in this digital world, a Server is a remote computer that provides information (data) or access to particular services.

So, it is the Client requesting something and the Server serving it, as long as it is in the **database**. As just discussed, the server often maintains a database, that is normally a separate entity logically, and could also be located on a different machine. Private data of the client can be stored in the browser, or as files that the browser has access to.

2. Web Attacks

2.1 Attacks on Server-Side Database

Server-side databases need to be protected, because the data stored there is long-lived and important (customer records, user records, etc.). We need to protect this data and the server-side state from illicit access and tampering.

A transaction is some expected interaction with the server, e.g., purchasing an item on a web store, or transferring money via a bank account. We need transactions to be compliant with the DBMS (Databased Management System) properties.

DBMS Properties:

In database systems, **ACID** (Atomicity, Consistency, Isolation, Durability) refers to a standard set of properties that guarantee database transactions are processed reliably. ACID is especially concerned with how a database recovers from any failure that might occur while processing a transaction. An ACID-compliant DBMS ensures that the data in the database remains accurate and consistent despite any such failures.

- **Atomicity** means that you guarantee that either all of the transaction succeeds or none of it does. You do not get part of it succeeding and part of it not. If one part of the transaction fails, the whole transaction fails. With atomicity, it's either "all or nothing".
- **Consistency** ensures that you guarantee that all data will be consistent. All data will be valid according to all defined rules, including any constraints, cascades, and triggers that have been applied on the database.
- **Isolation** guarantees that all transactions will occur in isolation. No transaction will be affected by any other transaction. So a transaction cannot read data from any other transaction that has not yet completed.
- **Durability** means that, once a transaction is committed, it will remain in the system – even if there is a system crash immediately following the transaction. Any changes from the transaction must be stored permanently. If the system tells the user that the transaction has succeeded, the transaction must have, in fact, succeeded.

SQL

Structured Query Language (SQL) is a domain-specific language used to manage data, especially in a relational database management system. It is particularly useful in handling structured data, i.e., data incorporating relations among entities and variables (i.e., in the form of tables).

SQL Injection

SQL Injection is a security flaw in web applications, where attackers insert harmful SQL code through user inputs. This can allow them to access sensitive data, change database contents or even take control of the system. Concretely the attacker is able to manipulate a web application's database queries by inserting malicious SQL code into user input fields. These injected queries can manipulate the underlying database to retrieve, modify, or delete sensitive data. In some cases, attackers can even escalate privileges, gaining full control over the database or server.

SQL Injection Countermeasures

- **Input Validation:** do NOT trust, but check user input!
 - **Check and Reject unexpected input – Whitelisting**
 - **Sanitize:**
 - Blacklisting: delete suspicious characters
 - Escaping: replace problematic characters
 - Prepared Statements

2.2 Maintaining the State

While the *long-lived* state is stored in the database, the web application also needs to maintain an *ephemeral* state. This is because we need to have multi-message exchanges. However, given the nature of the HTTP protocol, which is stateless, it is non-trivial to identify “the same client” and continue the request-response sequence. To facilitate this, the Servers (i.e., web applications) send the state to the client with each interaction, so that it can be used in the subsequent requests to identify the particular client/session.

- **Hidden Values:** A hidden input is just HTML sent to the client. It does not appear on the page to the end-user, but it is entirely accessible to the client. Meaning that the user can see it (just as they can see any HTTP response from your server) by using the View Source feature in their browser. This makes them abusable.
- **Capabilities:** Capabilities are unpredictable hidden values. The server maintains the state, and shares only unpredictable tokens (indexes into the state table) with the client. However, these are hard to maintain, as after closing the browser, the state (and as such also the capabilities) is lost.
- **Cookies:** Cookies are key-value pairs, which are part of the HTTP-headers, and are not exchanged in hidden HTML fields (as opposed to capabilities). The server maintains the state; the client receives the corresponding cookies, and stores them locally.
- **Attacks on Cookies:** A cookie can be issued upon user authentication, and be later used as a proof of authentication for subsequent sessions. As such, a stolen cookie can be used to perform session hijacking.

2.3 Server-Side Defenses

- Set sensitive cookies to be very short-lived
- Invalidate the cookie when the user logs-out
- Delete the cookie when the session ends
- Make cookies different from session to session
- Collect separate information and correlate whether the present interaction is a valid and legitimate request, based on past activity
 - monitoring the referrer fields, checking patterns, etc.

3. Attacks from Other Websites

Cross-Site Request Forgery (CSRF)

Cross-site request forgery (CSRF) is a type of attack where a malicious website tricks a user's browser into sending unauthorized commands to a web application that the user is logged into. This can lead

to unintended actions being performed on the user's account, such as changing settings or transferring funds, without their knowledge.

CSRF Protections:

- REFERER: The Server can check if the referer is the expected one, and if not, reject the request.
- Secret Link:
 - Part of the URL
 - HTTP header
 - Hidden form field
 - Cookie
- SameSite Cookie: SameSite prevents the browser from sending this cookie along with cross-site requests. The main goal is to mitigate the risk of cross-origin information leakage. It also provides some protection against cross-site request forgery attacks. Possible values for the flag are **none**, **lax**, or **strict**. The strict value will prevent the cookie from being sent by the browser to the target site in all cross-site browsing contexts, even when following a regular link. The lax value provides a reasonable balance between security and usability for websites that want to maintain user's logged-in session after the user arrives from an external link. The none value does not give any kind of protection. The browser attaches the cookies in all cross-site browsing contexts.

3.1 Client-Side Attacks

Dynamic Web Pages

JavaScript programs are executed in the browser and they can:

- alter web page contents (modifying the DOM)
- track user-side events (clicks, mouse movements, etc.)
- issue web requests and read replies
- access cookies and other browser data
- maintain persistent but invisible connections (AJAX)

As such, the web page does not visually change, but in the background javascript programs send web requests and do processing. Therefore, security issues arise.

Dynamic Web Pages – Secure

Browsers need to confine javascript programs so as to:

- ensure that scripts from one website cannot modify another webpage, or access its cookies
- ensure that the keystrokes of a user are not read, when typing passwords
- **Same Origin Policy (SOP):** All web page elements (layout, cookies, events, scripts) are associated with an origin. The origin is defined by the domain name (e.g., bank.com) that provided the web page (sent in the cookie also). Scripts can only access elements with the same origin as themselves.

Cross-Site Scripting (XSS) Attacks

Cross-Site Scripting (XSS) attacks are a type of injection, in which malicious scripts are injected into otherwise benign and trusted websites. XSS attacks occur when an attacker uses a web application to send malicious code, generally in the form of a browser side script, to a different end user. Flaws that allow these attacks to succeed are quite widespread and occur anywhere a web application uses input from a user within the output it generates without validating or encoding it.

- **Persistent XSS Attacks**

Stored or Persistent XSS generally occurs when user input is stored on the target server, such as in a database, in a message forum, visitor log, comment field, etc. And then a victim is able to retrieve the stored data from the web application without that data being made safe to render in the browser. With the advent of HTML5, and other browser technologies, we can envision the attack payload being permanently stored in the victim's browser, such as an HTML5 database, and never being sent to the server at all.

- **Reflected XSS Attacks**

Reflected XSS occurs when user input is immediately returned by a web application in an error message, search result, or any other response that includes some or all of the input provided by the user as part of the request, without that data being made safe to render in the browser, and without permanently storing the user provided data.